

Task 2 — QA Strategy & Thinking

Yasir Bari

Assumptions

The brief doesn't say what instruments the app supports, so I've assumed a standard trading app: deposit, buy/sell, withdraw. If leveraged products are involved, margin and liquidation accuracy move to the top of the list. I've also assumed a staging environment and test accounts with fake funds are available — if not, that's my first blocker.

1. Where do you start?

I'd start by figuring out where the money can break. Everything else is secondary.

Day one, I'd learn the app and test it at the same time. With 14 days to launch I can't spend two days just reading and talking. I'd install it on iOS and Android and use it like a real user: sign up, complete KYC, deposit, place an order, check balance and order history, withdraw. That surfaces real bugs within hours and teaches me the product faster than any document.

Alongside that, I'd talk to the developers and product owner — with no documentation, their heads are the documentation. What I'd want to know: how orders are executed and balances updated, which KYC provider and payment gateway are used and what happens when they fail or sit in pending, what they've already tested informally, and which parts of the app they're personally nervous about. Developers usually know where the weak spots are.

I'd also check what I have to work with: staging environment, test accounts with fake funds, API docs, and whether I can place test orders without touching real money. If I can't test safely, I'd raise that immediately.

From there I'd write a short risk list and get the team to agree on it. Given two weeks, my focus would be:

- **Money in and out** — deposit success, failure, stuck or pending, duplicate deposits, withdrawal limits, and the worst case of money leaving twice
- **Trading correctness** — placing and cancelling orders, correct price and quantity, balances updating correctly after every trade, order history matching reality
- **Auth and access** — login, session expiry, KYC blocking trading and withdrawals until verified
- **Interruption cases** — network dropping or the app being killed mid-order, where one tap becomes two orders or a deposit disappears

On automation, I'd be upfront: in two weeks I'm not building a framework. That time is better spent finding bugs. At most a small smoke check on the core APIs so each daily build gets a sanity pass. Real automation starts after launch, once the app stops changing under me.

And the part that matters most: I'm not the person who guarantees the app is safe — I'm the person who gives the team an honest picture of the risk. At the end of two weeks I'd bring evidence: what I tested, what I found, what's still unknown. If the money-critical paths aren't solid, I'd recommend either delaying or launching in a contained way — small beta group, deposit caps, one region. With real user funds, shipping on time means nothing if someone loses money on day one.

2. How would you approach testing this app?

Risk-based and layered — deep where money is involved, lighter everywhere else.

Start exploratory, but structured. One area at a time, time-boxed, taking notes. Those notes become the first QA documentation the team has, plus a simple checklist of what must work per flow that can be repeated on every build.

Test the API, not just the UI. The UI might show “order placed” while the API returned an error, or show a balance that doesn't match the backend. I'd test with Postman alongside the app and compare both sides.

Try to break it. The devs have done the happy path. My value is in the edge cases — insufficient balance, zero or negative amounts, double-tapping buy, cancelling an order while it executes, withdrawing more than available, trading from two devices at once.

Test third-party failures. Payment gateways and KYC providers time out, return pending, or reject. I'd check the user gets a clear error and the money stays consistent. If staging can't simulate these, I'd ask for mocks — these paths can't ship untested.

Mobile-specific cases. Network drop mid-transaction, app backgrounded or force-killed during an order, incoming calls, push notifications, biometric login and session timeout.

Device coverage, kept realistic. A newer and older iPhone, two Android devices at different price points, oldest supported OS versions, and one small screen.

Basic security. Not a pen test, but: no sensitive data in logs or plain text, session expires properly, KYC can't be bypassed.

iOS vs Android side by side. Two codebases usually means inconsistencies — missing validation, different rounding, different error handling.

Clear bug reports and daily retesting. Steps, device, build version, expected vs actual, evidence, and priority tied to money impact. Each fix gets retested plus the checklist around it re-run — that daily repeat is my regression safety net until a real suite exists.

3. What does QA look like inside a sprint?

QA starts before a line of code is written, not at the end of the sprint.

Requirement understanding. I join grooming and planning, review requirements and designs, and check the acceptance criteria are complete and clear. Ambiguity gets raised immediately — a missing rule caught in a meeting costs nothing, the same thing caught in

testing costs a day of rework. I also stay on top of mid-sprint changes, since those quietly break coverage.

Test planning. I prepare a test plan with estimated hours covering requirement understanding, test case writing, execution, retesting and regression, shared upfront so testing time is accounted for in the sprint.

Test design. Positive and negative cases split across critical, major and minor, mapped back to the acceptance criteria in a test matrix so coverage is visible. Where the API is ready before the UI, I start API testing early.

Test execution. Smoke first to check the build is worth testing, then full execution. Bugs logged with steps, severity, environment, build version and evidence, covering both platforms.

Bug management and regression. Move the story status, triage with PM/BA/Dev, retest fixes, then run regression — scoped to impacted areas if time is tight.

Sign-off and reporting. QA Done once critical and major bugs are closed, with a short summary before release: pass percentage, open defects, risks, and any low priority issues we're knowingly leaving open.

Handling delays. If the build comes late, I flag it immediately rather than absorbing it. Then I prioritise smoke plus critical cases, focus regression on impacted areas, and clearly state what wasn't covered. I'd rather say an area is untested than rush silently and let a gap reach production.

4. What does your ideal regression suite look like?

Small, fast, and trusted. A big suite that takes hours and fails randomly gets ignored — and once it's ignored it's worse than nothing, because now there's false confidence.

Layered. A smoke suite of 15–20 core checks that runs on every build in minutes and answers whether the build is worth testing. Core regression on money-critical flows before every release. Full regression before major releases or nightly.

Weighted by risk. Deposits, withdrawals, orders, balance accuracy and KYC gating get deep coverage; profile settings and static screens get a light pass. Better 40 strong tests around money than 400 shallow ones.

Mostly API, thin UI layer. API tests are faster and more stable, and that's where correctness lives. UI automation stays thin, covering only critical journeys. That doesn't mean less UI testing overall — just less automation, since layout, redirects and biometrics are quicker for a human to verify.

Stable and independent. Each test sets up its own data. Zero tolerance for flaky tests, because they train the team to ignore red results.

Automated in CI, with clear failures. Runs on every build without someone remembering, and the report shows what broke and why so a developer can act without asking me to reproduce it.

Plus a manual checklist for payment redirects, KYC upload, biometrics, push notifications and real-device behaviour.

Kept alive. Every escaped production bug gets a regression test added; tests that stop adding value get removed.

In this scenario, none of this exists on day one. In two weeks I'd build only the smoke layer and a manual checklist. The rest comes over the first two to three months after launch.

5. What would keep you up at night?

The thing that worries me most isn't a bug I find — it's the one I don't, where a user loses money and doesn't get it back.

Money going wrong quietly. A crash is obvious. What scares me is a balance off by a small amount, a rounding issue on the fourth decimal, or a deposit that shows as credited but never settled. Nobody notices for days, and by then you're reconciling accounts manually.

Duplicate transactions from a double tap. Weak network, button doesn't respond, user taps again. Without backend protection that's two orders or two withdrawals from one intent.

Interrupted transactions. Network drops or the app is killed mid-order. If the app and backend disagree about whether it went through, the user's money is in limbo.

Third-party failures. Gateways and KYC providers going down or sitting in pending. Outside our control and hardest to test, so most likely to be undertested.

Security. Two weeks isn't enough for a proper review, and this app holds funds and KYC documents. I can cover the basics, but I'd say clearly that a real security review is still needed rather than assume it's covered.

Everything I haven't had time to test. The risk isn't only what I found — it's what I never looked at, and I'd want that to be a conscious team decision, not a discovery afterwards.

And the thing you can't undo. A mobile fix means an App Store review and users who may not update for days. A serious bug stays live far longer than anyone expects. That's why I'd push for a staged rollout — limited users, deposit and withdrawal caps, one region — plus a kill switch to disable trading or withdrawals instantly.

Honestly: with real user funds and two weeks, I can't promise the app is bug-free. What I can do is make sure the team knows where the risk sits, and that if something goes wrong the blast radius is small and we can stop it quickly.

Appendix A — Test Plan

Objective: establish enough coverage in two weeks to give the team an evidence-based view of whether the app is safe to release with real funds.

In scope: signup and login, KYC verification and gating, deposit, order placement and cancellation, balance accuracy, order history, withdrawal, session and basic security, both platforms, interruption and network-failure handling.

Out of scope:

Excluded	Reason
Full security / penetration audit	Needs a specialist; schedule separately
Performance and load testing	No time; needed before scaling users
Full device matrix	Reduced matrix used instead
Localisation, accessibility, minor settings screens	Low money impact; post-launch
Automated UI regression framework	Poor use of two weeks; smoke API checks only

Approach: structured exploratory testing, functional testing against a core-flow checklist, negative and edge cases on money paths, API testing via Postman, mobile-specific scenarios, cross-platform comparison, basic security checks, and third-party failure testing using sandbox or mocks.

Environment and data: staging mirroring production; test accounts with fake funds at several KYC states; sandbox or mock access for the payment gateway and KYC provider; API docs or a Postman collection built in week one.

Device coverage:

Platform	Devices	OS
iOS	1 current, 1 older (small screen)	Latest and oldest supported
Android	1 flagship, 1 mid/low-range	Latest and oldest supported

Entry criteria: build installable on staging, smoke passing, test accounts and funds available, acceptance criteria or a walkthrough available.

Exit criteria: critical and major defects closed and retested; smoke and core regression passing on the release build; all money paths verified end to end including failures; open issues documented and accepted; QA summary and go/no-go shared.

Schedule:

Days	Focus
1-2	Learn app while testing; exploratory pass; set up test data; raise blockers
3-6	Deep testing of money flows; negative and edge cases; API vs UI comparison
7-8	Third-party failure paths, KYC gating, security basics, interruption scenarios
9-10	Cross-platform and device coverage; retesting fixes
11-12	Smoke automation for core APIs; regression

Days	Focus
13–14	on critical flows Release-build verification; QA summary, risk report, go/no-go

Deliverables: core-flow checklist and test cases, defect reports with evidence, daily status, smoke API check set, and a final QA summary covering pass rate, open defects, untested areas and risks.

Assumptions: staging behaves like production for money movement; sandbox or mocks available for third parties; developers available for questions; builds arrive early enough to test — late builds reduce coverage and will be flagged, not absorbed.

Appendix B — Release Readiness Checklist

The point of this list is to take emotion out of the release decision — instead of asking whether the app feels ready, the team works through fixed items and signs off on what is and isn't done.

#	Item	Owner	Status
1	Critical and major defects closed and retested	QA	
2	Smoke and core regression passing on the release build	QA	
3	Deposit verified end to end — success, failure, pending, duplicate	QA	
4	Withdrawal verified end to end — success, failure, limits, duplicate	QA	
5	Order placement, cancellation, balance and history verified	QA	
6	KYC gating blocks trading and withdrawal until verified	QA	
7	Amount validation — zero, negative, decimal, above balance and limit	QA	
8	Verified on real iOS	QA	

#	Item	Owner	Status
9	and Android devices, not only emulators Oldest supported OS and one small-screen device verified	QA	
10	Network loss, backgrounding and force-kill mid- transaction verified	QA	
11	Payment gateway verified — timeout, decline, pending	QA / Dev	
12	KYC provider verified — rejection, timeout, resubmission	QA / Dev	
13	Failed or pending transactions never credit or debit incorrectly	QA / Dev	
14	Session expiry and logout invalidate the session	QA	
15	No sensitive data in logs or plain-text storage	QA / Dev	
16	KYC/AML requirements confirmed	Product	
17	Security review completed or accepted as a known risk	Eng Lead	
18	Crash reporting, monitoring and transaction alerting live	Dev	
19	Kill switch available and rollback plan documented	Dev	
20	Staged rollout configured — limited users, caps, one	Product	

#	Item	Owner	Status
21	region Support briefed on known issues and escalation path	Product	
22	QA summary shared — pass rate, open defects, untested areas, risks	QA	
23	Go / no-go recorded with sign-off from QA, Dev Lead and Product	All	

No-go criteria — release should not proceed if any of these are true:

- Any open critical defect affecting funds, balances, orders or withdrawals
- Any money-movement path not verified end to end
- Duplicate transaction protection unverified
- Smoke or core regression failing on the release build
- No kill switch or rollback path available
- Untested areas not documented and accepted by the team

Appendix C — Risk Matrix

Severity is derived from likelihood and impact. Anything rated High needs a mitigation in place or an explicit, recorded acceptance before release.

ID	Risk	Likelihood / Impact	Severity	Mitigation
R1	Duplicate order or withdrawal from a double tap on a weak network	Medium / Critical	High	Idempotency testing at API level; backend duplicate protection; withdrawal caps at launch
R2	Transaction interrupted — app and backend disagree on state	High / Critical	High	Recovery testing; verify reconciliation on relaunch; pending states visible to the user
R3	Incorrect	Medium /	High	Verify balances

ID	Risk	Likelihood / Impact	Severity	Mitigation
	balance, rounding or settlement error going unnoticed	Critical		via API after every operation; test decimal and boundary amounts
R4	Gateway timeout or pending — money debited but not credited	Medium / Critical	High	Sandbox testing of failure states; no credit on failure; alerting on stuck transactions
R5	KYC bypass allowing unverified users to trade or withdraw	Low / Critical	High	Gating tests at UI and API level; attempt direct API calls without verification
R6	No time for a proper security review	High / Critical	High	Cover basics in QA scope; schedule external review; escalate as accepted risk
R7	Untested areas and device combinations due to the timeline	High / Major	High	Risk-based prioritisation; document untested areas in the QA summary
R8	Mobile fixes need store review — a defect stays live for days	High / Major	High	Staged rollout, caps, one region, kill switch for affected features
R9	Staging doesn't fully mirror production for money movement	Medium / Critical	High	Confirm config parity; limited controlled verification in production after release
R10	Late builds compress testing	High / Major	Medium	Flag immediately;

ID	Risk	Likelihood / Impact	Severity	Mitigation
	time			prioritise smoke and critical cases; report coverage gaps
R11	No automation — regression is manual and slow	High / Major	Medium	Build a smoke check set; manual critical-flow checklist; expand after launch
R12	Requirements undocumented, coverage depends on verbal knowledge	High / Major	Medium	Capture flows in writing while testing; confirm with devs and product
R13	Behaviour differs between iOS and Android	Medium / Major	Medium	Run identical flows on both platforms and log differences as defects
R14	Price data stale on screen, user trades on a wrong price	Medium / Major	Medium	Verify refresh after backgrounding and network recovery; check staleness indicator

Overall: given the number of High-severity risks and the two-week window, I'd recommend a contained release rather than a full public launch — limited users, deposit and withdrawal caps, one region, and a kill switch ready. That keeps the blast radius small while still letting the team ship on schedule.